МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

ОРДЕНА ТРУДОВОГО КРАСНОГО ЗНАМЕНИ
ФЕДЕРАЛЬНОГО ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ»

(МТУСИ)

Кафедра «Корпоративные информационные системы»

ЛАБОРАТОРНАЯ РАБОТА № 5

по дисциплине «Прикладные информационные системы»
на тему:

СОЗДАНИЕ ФОРМЫ И ПРЕДСТАВЛЕНИЯ ДЛЯ НОВОГО ПОСТА

Выполнила: студентка группы БСТ2256
Журавлева О.А.

Студенческий билет № 3БСТ22158

Проверил: Сергеев А.Е.

Москва 2026

С О Д Е Р Ж А Н И Е

Цель работы **3**

Задачи **3**

Ход работы **4**

1 Создание формы для добавления новых статей **4**

1.1 Настройка URL для создания статьи 4

1.2 Создание представления create_post 4

1.3 Создание шаблона create_post.html 5

2 Стилизация страницы создания статьи **6**

2.1 Подключение CSS 6

2.2 CSS стили для формы 6

3 Проверка уникальности названия статьи **8**

3.1 Модификация представления create_post 8

3.2 Отображение ошибки в шаблоне 8

4 Тестирование работы формы **9**

Заключение **12**

Цель работы

Научиться создавать формы и представления для добавления новых записей в блог с использованием Django: обработка POST-запросов, валидация данных, сохранение объектов в базу данных, авторизация пользователей, стилизация страницы формы и реализация проверки уникальности заголовка.

Задачи

1. Создать папку lab5 и скопировать в неё проект blog из лабораторной работы №4.

2. Добавить новый URL article/new/ и представление create_post для создания статьи.

3. Реализовать проверку авторизации пользователя (только авторизованные могут создавать статьи).

4. Обработать GET-запрос – показать пустую форму.

5. Обработать POST-запрос – проверить заполненность полей, уникальность заголовка, сохранить статью и перенаправить на страницу новой статьи.

6. Создать шаблон create_post.html с формой, защищённой csrf_token.

7. Подключить CSS-стили в соответствии с макетом.

8. Добавить проверку уникальности названия статьи и отображение ошибки.

Ход работы

1 Создание формы для добавления новых статей

Перед выполнением лабораторной работы в папке lab5 создадим копию проекта blog из лабораторной работы 4.

1.1 Настройка URL для создания статьи

В файле urls.py добавим маршрут для создания новой статьи:

```
from django.contrib import admin
from django.urls import path, re_path
from articles.views import archive, get_article, create_post

urlpatterns = [
    path('', archive, name='archive'),
    re_path(r'^article/(?P<article_id>\d+)$', get_article, name='get_article'),
    path('article/new/', create_post, name='create_post'),
    path('admin/', admin.site.urls),
]
```

1.2 Создание представления create_post

В файле views.py добавим функцию create_post.

Проверим, авторизован ли пользователь (request.user.is_authenticated).

Если нет – вернём ошибку 404.

При GET-запросе отдадим пустую форму.

При POST-запросе получим данные, проверим их, создадим статью и перенаправим на страницу этой статьи.

```
def create_post(request):
    if not request.user.is_authenticated:
        raise Http404("Только авторизованные пользователи могут создавать статьи.")

    if request.method == "POST":
        title = request.POST.get("title", "").strip()
        text = request.POST.get("text", "").strip()
        errors = {}

        if not title:
            errors["title"] = "Пожалуйста, укажите название статьи."
        if not text:
            errors["text"] = "Пожалуйста, введите текст статьи."

        # Проверка уникальности названия
        if title and Article.objects.filter(title=title).exists():
            errors["title_unique"] = "Статья с таким названием уже существует."
```

```
        if errors:
            return render(request, 'create_post.html', {
                'form': {'title': title, 'text': text, 'errors': errors}
            })
        else:
            article = Article.objects.create(
                title=title,
                text=text,
                author=request.user
            )
            # Перенаправляем на страницу созданной статьи
            return redirect('get_article', article_id=article.id)
    else:
        # GET-запрос - показать пустую форму
        return render(request, 'create_post.html', {'form': {}})
```

1.3 Создание шаблона create_post.html

В папке articles/templates/ создадим файл create_post.html:

```
{% load static %}
<!DOCTYPE html>
<html>
    <head>
        <title>Создать статью</title>
        <link rel="stylesheet" href="{% static 'css/create_post.css' %}">
    </head>
    <body class="create-page">
        <div class="header">
            <img src="{% static 'images/logo.png' %}" alt="Логотип" class="logo">
        </div>
        <div class="content">
            <h1>Написать статью</h1>
            <form method="POST">{% csrf_token %}
                <div class="field">
                    <input type="text" name="title" placeholder="Название статьи"
                           value="{{ form.title }}">
                    {% if form.errors.title %}
                        <div class="error">{{ form.errors.title }}</div>
                    {% endif %}
                    {% if form.errors.title_unique %}
                        <div class="error">{{ form.errors.title_unique }}</div>
                    {% endif %}
                </div>
                <div class="field">
                    <textarea name="text" placeholder="Текст статьи">{{ form.text
}}</textarea>
                    {% if form.errors.text %}
                        <div class="error">{{ form.errors.text }}</div>
                    {% endif %}
                </div>
                <input type="submit" value="Сохранить">
            </form>
        </div>
    </body>
</html>
```

2 Стилизация страницы создания статьи

2.1 Подключение CSS

1. Создадим файл articles/static/css/create_post.css.

2. Подключим его в шаблоне create_post.html

```
    <head>
        <title>Создать статью</title>
        <link rel="stylesheet" href="{% static 'css/create_post.css' %}">
    </head>
```

2.2 CSS стили для формы

В create_post.css пропишем стили в соответствии с макетом:

```
/* Общие стили */
body.create-page {
    background: #1abc9c;
    font-family: Tahoma, Arial, sans-serif;
    color: #ffffff;
    margin: 0;
    padding: 0;
}

/* Логотип */
.header img {
    display: block;
    width: 318px;
    margin: 20px auto;
}

/* Контейнер формы */
.content {
    width: 600px;
    margin: 0 auto;
    background: rgba(0,0,0,0.2);
    padding: 30px;
    border-radius: 10px;
}

h1 {
    text-align: center;
    margin-bottom: 30px;
}

/* Поля формы */
.field {
    margin-bottom: 20px;
}

input[type="text"], textarea {
    width: 100%;
    padding: 10px;
    font-size: 16px;
```

```
    border: none;
    border-radius: 5px;
    box-sizing: border-box;
}

textarea {
    height: 200px;
    resize: vertical;
}

input[type="submit"] {
    background: #ffffff;
    color: #1abc9c;
    border: none;
    padding: 10px 20px;
    font-size: 18px;
    border-radius: 5px;
    cursor: pointer;
    font-weight: bold;
}

input[type="submit"]:hover {
    background: #f0f0f0;
}

/* Ошибки */
.error {
    color: #ffcccc;
    font-size: 14px;
    margin-top: 5px;
}
```

3 Проверка уникальности названия статьи

3.1 Модификация представления create_post

В функции create_post после получения title добавим проверку:

```
# Проверка уникальности названия
if title and Article.objects.filter(title=title).exists():
    errors["title_unique"] = "Статья с таким названием уже существует."
```

3.2 Отображение ошибки в шаблоне

В create_post.html добавим блок:

```
{% if form.errors.title_unique %}
    <div class="error">{{ form.errors.title_unique }}</div>
{% endif %}
```

При попытке создать статью с существующим названием пользователь увидит сообщение об ошибке, а форма не отправится.

4 Тестирование работы формы

1. Запустим сервер разработки и перейдём на главную страницу http://127.0.0.1:8000/article/new/ (рис. 1):

```
python manage.py runserver
```

2. Проверка работы формы (рис. 2-7):

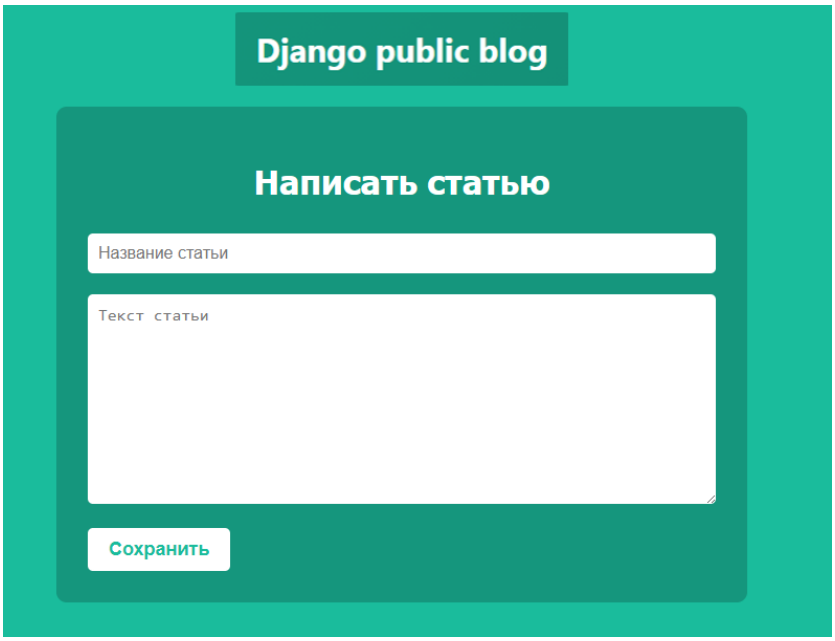

Рисунок 1 – страница http://127.0.0.1:8000/article/new/.

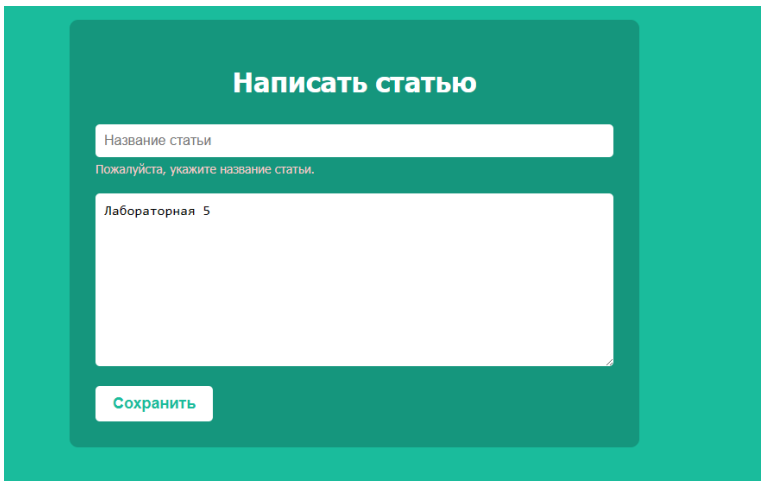

Рисунок 2 – попытка создания статьи без названия.

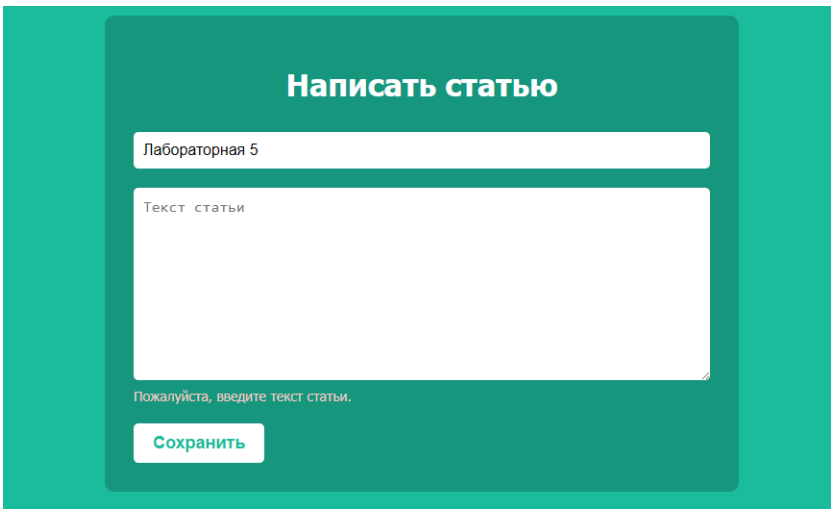

Рисунок 3 – попытка создания статьи без содержания.

Рисунок 4 – страница после создания статьи.

Рисунок 5 – страница со списком всех статей.

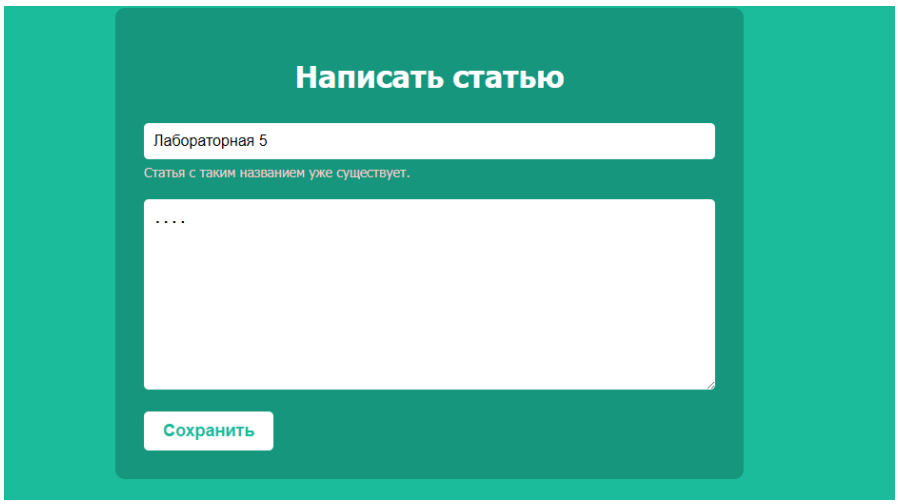

Рисунок 6 – попытка создания статьи с одинаковым названием.

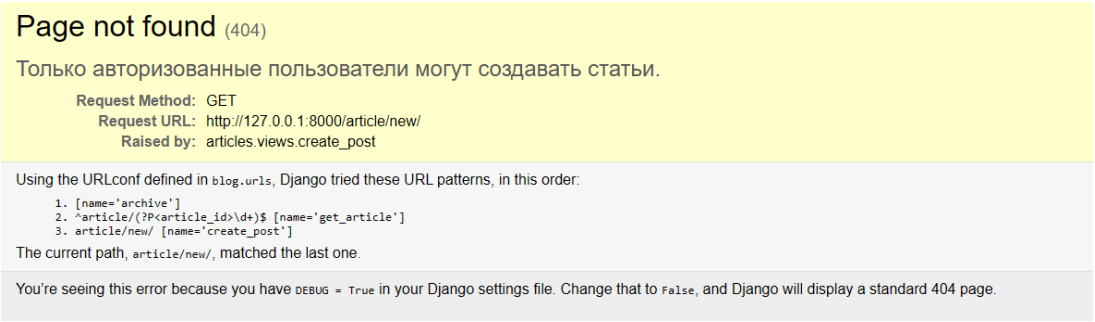

Рисунок 7 – переход на страницу без авторизации.

Заключение

В ходе лабораторной работы были выполнены все поставленные задачи: добавлен URL article/new/ и представление create_post с обработкой GET и POST запросов, реализована проверка авторизации – только зарегистрированные пользователи могут создавать статьи. Создан шаблон create_post.html с формой, защищённой CSRF, с отображением ошибок валидации. Подключены и настроены CSS-стили в соответствии с макетом lab5_creation_form. Добавлена проверка уникальности названия статьи: при попытке создать дубликат выводится ошибка. Выполнено тестирование – новые статьи успешно сохраняются, перенаправление работает.